

Nama	Muhammad Abyan Ridhan Siregar
NIM	1103210053
Kelas	TK-45-01

Machine Learning Task Week-14

RNN and Deep RNN Model

Part 1 : Import Library

```
import numpy as np
import torch
import torch.nn as nn
import torch.optim as optim
import torch.nn.functional as F

from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split

import re
import string
import os
```

Penjelasan

- Mengimpor berbagai pustaka yang dibutuhkan:
 - **Numpy** untuk operasi numerik dasar.
 - **Torch** (PyTorch) untuk membangun model deep learning.
 - **sklearn.model_selection** untuk membagi dataset (train/val).
 - **re, string, os** untuk keperluan *string manipulation* dan pengaturan path.

Part 2 : Data Preparation

```
def load_dataset(file_path):
    """
    Membaca file imdb_labelled.txt
    Format tiap baris: <teks>\t<label>
    """
    texts = []
    labels = []
    with open(file_path, 'r', encoding='utf-8') as f:
        for line in f:
```

```

# Pecah berdasarkan tab
line_split = line.strip().split('\t')
if len(line_split) == 2:
    text, label = line_split
    texts.append(text)
    labels.append(int(label))
return texts, labels

```

```

def text_cleaning(text):
    """
    Membersihkan teks sederhana: lower, hapus angka & tanda baca, dsb.
    """
    text = text.lower()
    text = re.sub(r'[%s]' % re.escape(string.punctuation), '', text) # hapus tanda baca
    text = re.sub(r'\d+', '', text) # hapus angka
    text = re.sub(r'\s+', ' ', text) # hapus spasi berlebih
    text = text.strip()
    return text

```

```

def build_vocab(cleaned_texts, min_freq=1):
    """
    Membuat word2idx sederhana berdasarkan frekuensi kemunculan kata.
    min_freq = minimal frekuensi agar kata dimasukkan ke vocab
    """
    freq_dict = {}
    for text in cleaned_texts:
        for word in text.split():
            freq_dict[word] = freq_dict.get(word, 0) + 1

    # sort by frequency
    sorted_words = sorted(freq_dict.items(), key=lambda x: x[1], reverse=True)

    # Buat word2idx, sisipkan token khusus <PAD> dan <UNK>
    word2idx = {'<PAD>':0, '<UNK>':1}
    idx = 2
    for word, freq in sorted_words:
        if freq >= min_freq:
            word2idx[word] = idx

```

<pre> idx += 1 return word2idx </pre>
<pre> def encode_text(text, word2idx, max_len=50): """ Mengubah teks menjadi list of token-id (integer), dan memotong/padding sampai max_len. """ tokens = text.split() encoded = [] for token in tokens: encoded.append(word2idx.get(token, word2idx['<UNK>'])) # potong jika panjang > max_len encoded = encoded[:max_len] # padding jika panjang < max_len if len(encoded) < max_len: encoded += [word2idx['<PAD>']] * (max_len - len(encoded)) return encoded </pre>
<pre> class IMDBDataset(Dataset): def __init__(self, texts, labels, word2idx, max_len=50): self.texts = texts self.labels = labels self.word2idx = word2idx self.max_len = max_len def __len__(self): return len(self.texts) def __getitem__(self, idx): text = self.texts[idx] label = self.labels[idx] encoded_text = encode_text(text, self.word2idx, self.max_len) return torch.LongTensor(encoded_text), torch.LongTensor([label]) # (seq), (label) </pre>
Penjelasan : • load_dataset(file_path): Membaca file dataset (misalnya imdb_labelled.txt) yang berisi teks dan label, lalu menyimpannya ke dalam list texts dan labels.

- **text_cleaning(text)**: Melakukan pembersihan teks sederhana (mengubah huruf menjadi *lowercase*, menghapus tanda baca, angka, dan spasi berlebih).
- **build_vocab(cleaned_texts, min_freq=1)**: Membuat *dictionary* (*word2idx*) yang memetakan kata ke indeks. Kata yang muncul kurang dari *min_freq* dapat diabaikan.
- **encode_text(text, word2idx, max_len=50)**: Mengubah teks yang sudah dibersihkan menjadi deretan indeks (*token IDs*). Juga dilakukan pemotongan jika panjang melebihi *max_len* dan *padding* jika kurang dari *max_len*.
- **IMDBDataset(Dataset)**: Kelas khusus untuk PyTorch yang mengelola data agar siap di-*load* oleh DataLoader. Setiap item *dataset* mengembalikan sepasang (*encoded_text*, *label*) dalam bentuk *torch.Tensor*.

Part 3 :Model Definition

```
class RNNClassifier(nn.Module):
    def __init__(self, vocab_size, embed_dim=128, hidden_size=128, num_layers=1,
                  pooling='max', num_classes=2):
        super(RNNClassifier, self).__init__()
        self.embedding = nn.Embedding(vocab_size, embed_dim, padding_idx=0)

        # RNN (vanilla RNN); jika ingin LSTM/GRU ganti di sini
        self.rnn = nn.RNN(input_size=embed_dim,
                          hidden_size=hidden_size,
                          num_layers=num_layers,
                          batch_first=True,
                          nonlinearity='tanh',
                          dropout=0.0,
                          bidirectional=False)

        self.pooling = pooling # 'max' or 'avg'
        self.fc = nn.Linear(hidden_size, num_classes)

    def forward(self, x):
        # x shape: (batch_size, seq_len)
        embedded = self.embedding(x) # (batch_size, seq_len, embed_dim)
        rnn_out, h_n = self.rnn(embedded) # (batch_size, seq_len, hidden_size)
```

```

if self.pooling == 'max':
    # MaxPooling over seq_len
    pooled, _ = torch.max(rnn_out, dim=1) # (batch_size, hidden_size)
else:
    # AvgPooling
    pooled = torch.mean(rnn_out, dim=1) # (batch_size, hidden_size)

logits = self.fc(pooled) # (batch_size, num_classes)
return logits

```

Penjelasan

- **RNNClassifier**: Kelas model utama yang memanfaatkan arsitektur *vanilla* RNN.
 - **nn.Embedding**: Mengubah indeks kata menjadi *embedding vector* dengan dimensi `embed_dim`.
 - **nn.RNN**: Modul RNN untuk memproses sekuens embedding. Bisa diganti dengan LSTM/GRU sesuai kebutuhan.
 - **Pooling**:
 - *Max Pooling* mengambil nilai maksimum di sepanjang *sequence length*.
 - *Average Pooling* mengambil nilai rata-rata di sepanjang *sequence length*.
 - **fc (nn.Linear)**: *Fully-connected layer* terakhir yang mengubah dimensi *hidden state* menjadi jumlah kelas (2 untuk *binary classification*).

Part 4 : Training & Eval Utils

```

def calculate_accuracy(y_pred, y_true):
    """
    Hitung akurasi antara prediksi dan label sebenarnya.
    """
    predicted = torch.argmax(y_pred, dim=1)
    correct = (predicted == y_true.view(-1)).sum().item()
    total = y_true.size(0)
    return correct / total

```

```

def train_one_epoch(model, dataloader, optimizer, criterion, device='cpu'):
    model.train()
    running_loss = 0.0
    running_acc = 0.0
    for x_batch, y_batch in dataloader:

```

```
x_batch, y_batch = x_batch.to(device), y_batch.to(device)
```

```
optimizer.zero_grad()
```

```
outputs = model(x_batch)
```

```
loss = criterion(outputs, y_batch.view(-1))
```

```
loss.backward()
```

```
optimizer.step()
```

```
acc = calculate_accuracy(outputs, y_batch)
```

```
running_loss += loss.item() * x_batch.size(0)
```

```
running_acc += acc * x_batch.size(0)
```

```
epoch_loss = running_loss / len(dataloader.dataset)
```

```
epoch_acc = running_acc / len(dataloader.dataset)
```

```
return epoch_loss, epoch_acc
```

```
def validate_one_epoch(model, dataloader, criterion, device='cpu'):
```

```
    model.eval()
```

```
    running_loss = 0.0
```

```
    running_acc = 0.0
```

```
    with torch.no_grad():
```

```
        for x_batch, y_batch in dataloader:
```

```
            x_batch, y_batch = x_batch.to(device), y_batch.to(device)
```

```
            outputs = model(x_batch)
```

```
            loss = criterion(outputs, y_batch.view(-1))
```

```
            acc = calculate_accuracy(outputs, y_batch)
```

```
            running_loss += loss.item() * x_batch.size(0)
```

```
            running_acc += acc * x_batch.size(0)
```

```
    epoch_loss = running_loss / len(dataloader.dataset)
```

```
    epoch_acc = running_acc / len(dataloader.dataset)
```

```
    return epoch_loss, epoch_acc
```

Penjelasan

- **calculate_accuracy(y_pred, y_true):** Menghitung akurasi, di mana label prediksi diambil dari argmax pada y_pred.
- **train_one_epoch(...):**
 - Mengaktifkan mode *train* (model.train()).

- Melakukan loop di seluruh batch untuk *forward pass*, menghitung *loss*, *backward pass*, dan `optimizer.step()`.
- Mencatat *loss* dan *accuracy* di setiap batch, lalu dirata-rata.
- **validate_one_epoch(...):**
 - Mengaktifkan mode *eval* (`model.eval()`).
 - **Tanpa** *backward pass*, hanya menghitung *forward* dan *loss* serta akurasi di *validation set*.

Part 5 : Early Stopping Class

```
class EarlyStopping:
    def __init__(self, patience=5, delta=0.0):
        self.patience = patience
        self.counter = 0
        self.best_loss = None
        self.early_stop = False
        self.delta = delta

    def __call__(self, val_loss):
        if self.best_loss is None:
            self.best_loss = val_loss
        elif val_loss > self.best_loss - self.delta:
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_loss = val_loss
            self.counter = 0
```

Penjelasan

- **EarlyStopping:** Menghentikan proses training jika *loss* pada *validation set* tidak membaik setelah beberapa *epoch* tertentu (*patience*).
- **self.delta:** Toleransi perbedaan minimal agar dianggap “membaik”.
- **self.counter:** Menghitung berapa kali *validation loss* tidak mengalami penurunan.

Part 6 : Main Training Function

```
def train_and_evaluate(model,
                        train_loader,
                        val_loader,
                        optimizer,
                        scheduler,
                        criterion,
                        epochs=10,
                        patience=5,
                        device='cpu'):

    early_stopper = EarlyStopping(patience=patience)

    best_val_loss = float('inf')
    best_model_state = None

    for epoch in range(1, epochs+1):
        # Training
        train_loss, train_acc = train_one_epoch(model, train_loader, optimizer, criterion, device)
        # Validation
        val_loss, val_acc = validate_one_epoch(model, val_loader, criterion, device)

        # Scheduler step (misalnya StepLR atau lainnya)
        if scheduler is not None:
            scheduler.step(val_loss) # Jika pakai ReduceLROnPlateau, butuh param (val_loss)

        print(f'Epoch [{epoch}/{epochs}] | "
              f"Train Loss: {train_loss:.4f}, Train Acc: {train_acc:.4f} | "
              f"Val Loss: {val_loss:.4f}, Val Acc: {val_acc:.4f}")

        # Cek apakah ini model terbaik
        if val_loss < best_val_loss:
            best_val_loss = val_loss
            best_model_state = model.state_dict()

    # Early Stopping
```



```

early_stopper(val_loss)
if early_stopper.early_stop:
    print("Early stopping triggered!")
    break

# Kembalikan state terbaik
if best_model_state is not None:
    model.load_state_dict(best_model_state)
return model

```

Penjelasan

- **train_and_evaluate(...):**
 1. Inisialisasi **Early Stopping**.
 2. Loop per-epoch:
 - Panggil `train_one_epoch` untuk melatih model di *train set*.
 - Panggil `validate_one_epoch` untuk mengevaluasi model di *validation set*.
 - Update *learning rate* melalui *scheduler* (misal `ReduceLROnPlateau`) berdasarkan *validation loss*.
 - Cek **Early Stopping**: jika *loss* tidak membaik setelah *patience* tertentu, hentikan training.
 3. Jika ada model dengan *val loss* terbaik, simpan *state_dict* dan kembalikan model terbaik di akhir.

Part 7 : Experiment Config

```

from google.colab import drive
import pandas as pd
from google.colab import files
drive.mount('/content/drive')

FILE_PATH = "/content/drive/MyDrive/Machine Learning/imdb_labelled.txt" # sesuaikan dengan
path dataset
BATCH_SIZE = 32
MAX_LEN = 50
EMBED_DIM = 128
DEVICE = 'cpu' # jika ingin pakai GPU: DEVICE = 'cuda' (jika tersedia)

```

```
# Hyper-parameters yang mau kita bandingkan
```

```
HIDDEN_SIZES = [64, 128]
```

```
POOLINGS = ['max', 'avg']
```

```
OPTIMIZERS = ['SGD', 'RMSProp', 'Adam']
```

```
EPOCHS_LIST = [5, 50, 100, 250, 350]
```

```
# Callback/EarlyStop param
```

```
PATIENCE = 5
```

```
results = []
```

Penjelasan

- Di sini ditentukan berbagai **parameter** untuk eksperimen, seperti:
 - **FILE_PATH**: lokasi dataset.
 - **BATCH_SIZE**: ukuran batch.
 - **MAX_LEN**: panjang sekuens maksimum setelah *padding*.
 - **HIDDEN_SIZES**: daftar opsi *hidden size* RNN yang akan dicoba.
 - **POOLINGS**: jenis pooling (max atau avg).
 - **OPTIMIZERS**: opsi optimizer (SGD, RMSProp, Adam).
 - **EPOCHS_LIST**: daftar jumlah *epoch* yang akan dicoba.
 - **PATIENCE**: kesabaran untuk *Early Stopping*.

Part 8 : Main Experiment Script

```
def main_experiment_with_logging():
```

```
    # 1. Load data
```

```
    texts, labels = load_dataset(FILE_PATH)
```

```
    # 2. Bersihkan teks
```

```
    cleaned_texts = [text_cleaning(t) for t in texts]
```

```
    # 3. Build vocab
```

```
    word2idx = build_vocab(cleaned_texts, min_freq=1)
```

```
    vocab_size = len(word2idx)
```

```
    # 4. Train-Val split
```

```
    train_texts, val_texts, train_labels, val_labels = train_test_split(
```

```
        cleaned_texts, labels, test_size=0.2, random_state=42
```

)

5. Buat dataset & dataloader

```
train_dataset = IMDBDataset(train_texts, train_labels, word2idx, max_len=MAX_LEN)
```

```
val_dataset = IMDBDataset(val_texts, val_labels, word2idx, max_len=MAX_LEN)
```

```
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True)
```

```
val_loader = DataLoader(val_dataset, batch_size=BATCH_SIZE, shuffle=False)
```

6. Criterion

```
criterion = nn.CrossEntropyLoss()
```

7. Kita akan menyimpan hasil di list "results"

```
results = [] # akan menampung dict di setiap kombinasi
```

8. Looping sesuai settingan experiment

```
for hidden_size in HIDDEN_SIZES:
```

```
    for pooling in POOLINGS:
```

```
        for opt_name in OPTIMIZERS:
```

```
            for epochs in EPOCHS_LIST:
```

```
                print("=====")
```

```
                print(f'[INFO] Training with hidden_size={hidden_size}, pooling={pooling}, "
```

```
                    f'optimizer={opt_name}, epochs={epochs}')
```

```
            # Buat model baru
```

```
            model = RNNClassifier(
```

```
                vocab_size=vocab_size,
```

```
                embed_dim=EMBED_DIM,
```

```
                hidden_size=hidden_size,
```

```
                num_layers=1, # ubah jika mau lebih deep
```

```
                pooling=pooling,
```

```
                num_classes=2
```

```
            ).to(DEVICE)
```

```
            # Buat optimizer
```

```
            if opt_name == 'SGD':
```

```

        optimizer = optim.SGD(model.parameters(), lr=0.01)
    elif opt_name == 'RMSProp':
        optimizer = optim.RMSprop(model.parameters(), lr=0.001)
    else: # Adam
        optimizer = optim.Adam(model.parameters(), lr=0.001)

    # Scheduler (contoh: ReduceLROnPlateau)
    scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.5,
patience=2)

    # Train & Evaluate (dengan EarlyStopping)
    trained_model = train_and_evaluate(
        model,
        train_loader,
        val_loader,
        optimizer,
        scheduler,
        criterion,
        epochs=epochs,
        patience=PATIENCE,
        device=DEVICE
    )

    # Setelah training selesai (atau early stop),
    # kita ukur final performance di train set & val set
    final_train_loss, final_train_acc = validate_one_epoch(trained_model, train_loader,
criterion, DEVICE)

    final_val_loss, final_val_acc = validate_one_epoch(trained_model,
val_loader, criterion, DEVICE)

    # Simpan ke "results"
    results.append({
        "hidden_size": hidden_size,
        "pooling": pooling,
        "optimizer": opt_name,
        "epochs": epochs,

```

```

        "final_train_loss": final_train_loss,
        "final_train_acc": final_train_acc,
        "final_val_loss": final_val_loss,
        "final_val_acc": final_val_acc
    })

    print("=====\\n")

# 9. Buat DataFrame dari results
df_results = pd.DataFrame(results)

# 10. Simpan ke CSV
df_results.to_csv('experiment_results.csv', index=False)
print("Hasil eksperimen telah disimpan ke 'experiment_results.csv'.")

# 11. Download CSV ke lokal (khusus untuk Colab)
files.download('experiment_results.csv')
print("File CSV didownload ke komputer lokal Anda.")

# Jalankan experiment
if __name__ == "__main__":
    main_experiment_with_logging()

```

Penjelasan

- Di sini ditentukan berbagai **parameter** untuk eksperimen, seperti:
 - **FILE_PATH**: lokasi dataset.
 - **BATCH_SIZE**: ukuran batch.
 - **MAX_LEN**: panjang sekuens maksimum setelah *padding*.
 - **HIDDEN_SIZES**: daftar opsi *hidden size* RNN yang akan dicoba.
 - **POOLINGS**: jenis pooling (max atau avg).
 - **OPTIMIZERS**: opsi optimizer (SGD, RMSProp, Adam).
 - **EPOCHS_LIST**: daftar jumlah *epoch* yang akan dicoba.
 - **PATIENCE**: kesabaran untuk *Early Stopping*.

Analisis Hasil :

Dari hasil yang di dapat pada training di atas dan hasil result csv dapat disimpulkan bahwa

1. Pengaruh *Hidden Size*

Hidden size yang lebih besar (128) pada umumnya memberikan representasi lebih kaya terhadap pola sekuens, sehingga performa (terutama akurasi pada data validasi) cenderung meningkat dibandingkan *hidden size* yang lebih kecil (64). Namun, kelebihan *hidden size* juga dapat memakan waktu komputasi lebih lama dan berisiko *overfitting* jika jumlah data tidak mencukupi.

2. Perbandingan *Pooling* (*Max* vs. *Avg*)

Pada beberapa kasus, *Max Pooling* bisa jadi unggul karena lebih menekankan *feature* paling menonjol di setiap dimensi keluaran RNN. *Avg Pooling* cenderung merata, sehingga performa tidak jauh berbeda, tapi ada kalanya *avg* menghasilkan *loss* yang sedikit lebih rendah karena lebih “stabil”. Hasil menunjukkan bahwa tidak ada satu jenis *pooling* yang mendominasi secara mutlak, meskipun *Max Pooling* sering kali sedikit unggul pada akurasi.

3. Pengaruh *Optimizer*

- *Adam* cenderung memiliki konvergensi yang lebih cepat dan stabil. Hal ini terlihat dari kemampuan Adam mencapai akurasi tertinggi dalam *epoch* yang lebih singkat. *RMSProp* kerap memiliki performa mendekati Adam, sedangkan *SGD* cenderung butuh *epoch* lebih banyak untuk memperoleh hasil yang sebanding.
- Jika dataset lebih besar, *SGD* dengan *learning rate* dan *momentum* yang tepat dapat menyamai bahkan melebihi Adam, tetapi untuk eksperimen singkat, Adam terlihat lebih praktis dalam mencapai hasil yang konsisten.

4. Jumlah *Epoch*

- Pada *epoch* sedikit (5, 50), model mungkin belum belajar optimal (terlihat dari akurasi yang lebih rendah).
- Sementara pada *epoch* lebih banyak (100, 250, 350), peningkatan performa semakin melambat dan risiko *overfitting* meningkat. *Early Stopping* cukup efektif memotong pelatihan pada *epoch* ke sekian saat *loss* validasi tidak membaik lagi.

Secara garis besar, hasil menunjukkan bahwa **Adam** dengan *hidden size* **128** sering memberikan akurasi yang relatif tinggi, terutama pada *pooling* **Max**. Meski begitu, setiap kombinasi memiliki kelebihan dan kekurangan masing-masing tergantung ketersediaan sumber daya dan data.

Kesimpulan :

Dari rangkaian eksperimen dalam *Week_14 Notebook*, dapat disimpulkan bahwa **model RNN** pada tugas klasifikasi sentimen umumnya diuntungkan oleh pemilihan *hidden size* yang lebih besar dan *optimizer* yang mampu beradaptasi cepat seperti Adam. Penambahan mekanisme *pooling* (Max atau

Avg) turut memengaruhi cara representasi *feature* digabungkan, tetapi tidak selalu menghasilkan perbedaan performa signifikan. Meski demikian, pemakaian *epoch* tinggi belum tentu lebih baik jika tidak diimbangi dengan *regularization* atau *early stopping*; hal ini ditunjukkan dari bagaimana performa validasi kadang stagnan atau menurun setelah beberapa *epoch*. Oleh karena itu, pemilihan *hyperparameter* yang tepat, pemantauan *training* secara berkala, dan penerapan callback seperti *Early Stopping* menjadi faktor kunci untuk memperoleh model dengan performa konsisten tanpa berlebihan dalam waktu dan sumber daya.